

CodeCrunch

CS2030 (2510) Practical Assessment #1

Course Prerequisites

This task was inspired from a HS2924 (Building Mental Wellbeing and Resilience) group project that aims to study the effectiveness of the Grade-Free (S/U) Scheme at NUS.

Course prerequisites are upstream courses that a student must fulfil before they are allowed to read another downstream course. This is to ensure that students have the necessary knowledge and skillset before embarking on the new course, and prevents them from entering courses they are not ready for.

Task

Your task is to design a software to check that a set of courses that a student has taken fulfil all course prerequisites. Different S/U schemes can also be introduced to trim down the list into an eventual list of graded courses.

Take Note!

This task comprises a number of levels. You are required to complete ALL levels.

The following are the constraints imposed on this task. In general, you should keep to the constructs and programming discipline instilled throughout the module.

- Write each class or interface in its own file. Do not use single letter names for classes or interfaces.
- Ensure that ALL object properties and class constants are declared `private final`
- Ensure that ALL classes/interfaces are NOT cyclic dependent.
- ONLY the following java libraries ARE allowed:
 - functional interfaces from `java.util.function`
- The following are NOT allowed:
 - `var`, `null`, `default`, `enum`
 - `instanceof` (except checking for an instance of its own class)
- You are NOT allowed to use bounded type parameters or bounded wildcards.
- You are NOT allowed to define anonymous inner classes or method references ::
Define lambdas instead.
- Other usual restrictions:
 - Use only `&&`, `||` and `!` in logical expressions. DO NOT use bitwise operators.
 - You are NOT allowed to use `*` wildcard imports.
 - You are NOT allowed to define array constructs, e.g. `String[]` or using ellipsis, e.g. `String...`
 - You are NOT allowed to perform string processing using methods from `String`, `StringBuffer`, `StringBuilder`, etc.
 - You are NOT allowed to use exception handling
 - You are NOT allowed to use Java reflection, i.e. `Object::getClasses` and other methods from `java.lang.Class`

In the following level specifications, you will be guided on the classes and non-private methods to define. DO NOT create any other classes. Moreover, any other method you create must be declared `private`.

The following classes has been provided for you. DO NOT replace them with your own versions.

- [Pair](#) class (not a record, but behaves similarly)
- [Maybe](#) class
- [InfList](#) class

To automatically generate HTML documentation from the comments, issue the command:

```
$ javadoc -d doc InfList.java Maybe.java Pair.java
```

You may then navigate through the documentation from `allclasses-index.html` found in the `doc` directory.

Level 1

Write a `Course` class with a constructor that takes in the course code as a `String`. Include a meaningful string representation for the class, as well as the `equals` method. Two courses are the same as long as their course codes are the same. Note that course codes are case-sensitive.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> new Course("cs2030")
$.. ==> [cs2030]

jshell> new Course("cs2030").equals(new Course("cs2030"))
$.. ==> true

jshell> new Course("cs2030").equals(new Course("CS2030"))
$.. ==> false

jshell> new Course("cs2030").equals("cs2030")
$.. ==> false
```

Level 2

In this level, you are to implement a class `CourseReq` to represent a course with a prerequisite. `CourseReq` is still a course, but with an additional prerequisite (`Prereq`) that needs to be satisfied.

You are given the following [Prereq](#) interface. DO NOT replace this with your own version.

```
interface Prereq {
    public boolean satisfiedBy(InfList<Course> courses);
}
```

Note that a `CourseReq` object may also be a prerequisite for some other `CourseReq` object.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> CourseReq cs1010 = new CourseReq("cs1010") // course with no prerequisite
cs1010 ==> [cs1010]

jshell> CourseReq cs2030 = new CourseReq("cs2030", cs1010) // course with a prerequisite
cs2030 ==> [cs2030]

jshell> CourseReq cs2040 = new CourseReq("cs2040", cs1010) // course with a prerequisite
cs2040 ==> [cs2040]
```

Following the specifications of the `Prereq` interface, implement the `satisfiedBy` method that takes in a list of courses of type `InfList<Course>`, and returns `true` if the prerequisites are satisfied, and `false`

otherwise.

In this level, only courses with at most one prerequisite course are tested. Moreover to simplify testing, `InfList` now includes an `of` method that takes in elements separated by commas.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> cs1010.satisfiedBy(InfList.<Course>of(cs1010))
$.. ==> true

jshell> cs1010.satisfiedBy(InfList.<Course>of(cs1010, cs2030))
$.. ==> true

jshell> cs2030.satisfiedBy(InfList.<Course>of(cs1010)) // need to also read cs2030
$.. ==> false

jshell> cs2030.satisfiedBy(InfList.<Course>of(cs1010, cs2030))
$.. ==> true

jshell> cs2030.satisfiedBy(InfList.<Course>of(cs2030)) // need cs1010 as a prerequisite
$.. ==> false

jshell> cs2030.satisfiedBy(InfList.<Course>of(cs1010, cs2040)) // need to read cs2030
$.. ==> false

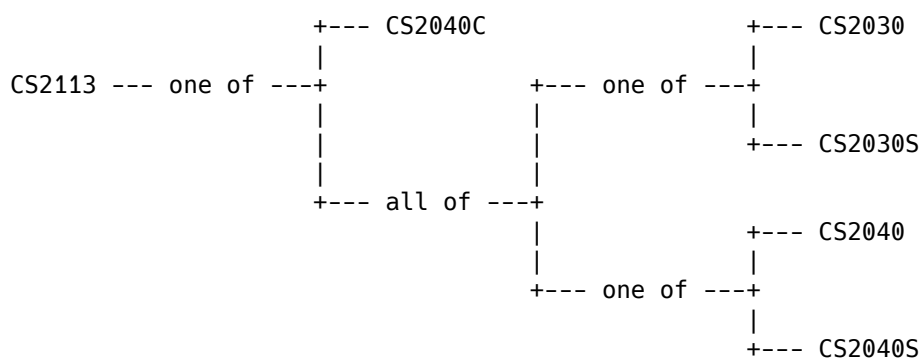
jshell> cs2040.satisfiedBy(InfList.<Course>of(cs1010, cs2030, cs2040))
$.. ==> true

jshell> cs2030.satisfiedBy(InfList.<Course>of(cs2030, cs1010))
$.. ==> true
```

The last test case shows that the list of courses need not be in the order of the time they are read, so long as they meet the prerequisites, if any.

Level 3

Let us take a look at another more interesting example. A simplified version of the prerequisite for CS2113 is given below:



Notice there are two general types of prerequisites: *one of* and *all of*. Each one of these is a constraint over other prerequisites (can be *one of*, *all of*, or courses with prerequisites).

By now, you should already have defined the courses with prerequisites.

```
jshell> cs2030 instanceof Prereq
$.. ==> true

jshell> cs1010 instanceof Prereq
$.. ==> true
```

You will now need to write the `AllOf` and `OneOf` classes. For each of them, write a constructor that takes in a list of prerequisites. You will also need to define the `satisfiedBy` methods accordingly. Here are some examples.

```
$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> CourseReq cs1010 = new CourseReq("cs1010")
cs1010 ==> [cs1010]

jshell> CourseReq cs2030 = new CourseReq("cs2030", cs1010)
cs2030 ==> [cs2030]

jshell> CourseReq cs2030s = new CourseReq("cs2030s", cs1010)
cs2030s ==> [cs2030s]

jshell> CourseReq cs2040 = new CourseReq("cs2040", cs1010)
cs2040 ==> [cs2040]

jshell> Prereq oneOf2030 = new OneOf(InfList.of(cs2030, cs2030s))
oneOf2030 ==> OneOf@..

jshell> oneOf2030.satisfiedBy(InfList.of(cs1010))
$.. ==> false

jshell> oneOf2030.satisfiedBy(InfList.of(cs1010, cs2030))
$.. ==> true

jshell> oneOf2030.satisfiedBy(InfList.of(cs1010, cs2030s))
$.. ==> true

jshell> oneOf2030.satisfiedBy(InfList.of(cs1010, cs2030, cs2030s))
$.. ==> true

jshell> oneOf2030.satisfiedBy(InfList.of(cs2030, cs2030s)) // need cs1010 as a prerequisite
$.. ==> false

jshell> Prereq allOf2030_2040 = new AllOf(InfList.of(cs2030, cs2040))
allOf2030_2040 ==> AllOf@..

jshell> allOf2030_2040.satisfiedBy(InfList.of(cs2030))
$.. ==> false

jshell> allOf2030_2040.satisfiedBy(InfList.of(cs2040))
$.. ==> false

jshell> allOf2030_2040.satisfiedBy(InfList.of(cs2030, cs2040))
$.. ==> false

jshell> allOf2030_2040.satisfiedBy(InfList.of(cs2030, cs2040, cs1010))
$.. ==> true

jshell> allOf2030_2040.satisfiedBy(InfList.of(cs2030, cs1010))
$.. ==> false
```

Hint: `InfList` now includes the `anyMatch`, `allMatch` and `noneMatch` methods that you may find useful.

Here is the complete test case for the prerequisite of CS2113 shown above. To keep the example short, we assume that all variants of CS2030 and CS2040 only requires CS1010. That being said, more rigorous testing will be done where, for example, CS2030 has a prerequisite of one of CS1010, CS1010a, CS1010e, CS1010s, etc.

```
jshell> CourseReq cs2040s = new CourseReq("cs2040s", cs1010)
cs2040s ==> [cs2040s]

jshell> CourseReq cs2040c = new CourseReq("cs2040c", cs1010)
cs2040c ==> [cs2040c]
```

```

jshell> CourseReq cs2113 = new CourseReq("cs2113",
...> new OneOf(InfList.of(cs2040c,
...> new AllOf(InfList.of(
...> new OneOf(InfList.of(cs2030, cs2030s)),
...> new OneOf(InfList.of(cs2040, cs2040s))))))
cs2113 ==> [cs2113]

jshell> cs2113.satisfiedBy(InfList.of(cs2040c))
$.. ==> false

jshell> cs2113.satisfiedBy(InfList.of(cs2040c, cs1010))
$.. ==> false

jshell> cs2113.satisfiedBy(InfList.of(cs2040c, cs1010, cs2113))
$.. ==> true

jshell> cs2113.satisfiedBy(InfList.of(cs2040, cs1010, cs2113))
$.. ==> false

jshell> cs2113.satisfiedBy(InfList.of(cs2040, cs1010, cs2113, cs2030))
$.. ==> true

jshell> cs2113.satisfiedBy(InfList.of(cs2040, cs1010, cs2113, cs2030s))
$.. ==> true

```

Level 4

Write a `CoursePlan` class with a `constructor` that takes in a `list of courses with prerequisites`, and creates a `viable plan` by ignoring all courses that do not meet the prerequisites. Include an appropriate `toString` method to represent all remaining courses, or "no courses" if there are no courses left.

```

$ javac your_java_files
$ jshell your_java_files_in_bottom-up_dependency_order

jshell> new CoursePlan(InfList.of())
$.. ==> no courses

jshell> new CoursePlan(InfList.of(cs1010))
$.. ==> [cs1010]

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040))
$.. ==> [cs1010][cs2030][cs2040]

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113))
$.. ==> [cs1010][cs2030][cs2040][cs2113]

jshell> new CoursePlan(InfList.of(
...> cs1010, cs2030, cs2113)) // cs2113 dropped due to missing cs2040/cs2040s/cs2040c
$.. ==> [cs1010][cs2030]

jshell> new CoursePlan(InfList.of(
...> cs2030, cs2040, cs2113)) // all courses dropped due to missing cs1010
$.. ==> no courses

```

Note from the above examples that courses are displayed in order of presentation.

Now include a `numOfCourses` method that returns the number of remaining courses in the course plan. Also include a `drop` method that takes in a course and drops the course from the plan (possibly due to academic dishonesty that was found out later) and rework the courses to meet the prerequisites.

```

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113)).numOfCourses()
$.. ==> 4

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113)).drop(cs2113)
$.. ==> [cs1010][cs2030][cs2040]

```

```
jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113)).drop(cs2113).
...> numOfCourses()
$.. ==> 3

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113)).drop(cs1010)
$.. ==> no courses

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113)).drop(cs1010).
...> numOfCourses()
$.. ==> 0

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040)).drop(cs2113)
$.. ==> [cs1010][cs2030][cs2040]

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040)).drop(cs2113).numOfCourses()
$.. ==> 3

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040)).drop(new Course("cs2040")).
...> numOfCourses()
$.. ==> 2
```

Level 5

In the `CoursePlan` class, include a `trimSU` method that returns a list of courses of type `InfList<Course>` after trimming all S/U'ed courses (i.e. courses with satisfactory/unsatisfactory grading). Different policies for S/U can be provided to `trimSU`, e.g.

- courses at the start of the prerequisite chain can be S/U'ed; or
- courses at the end of the prerequisite chain can be S/U'ed, i.e. courses that are not prerequisites of other courses.

I personally find this a more meaningful policy to help build mental well-being (mine included) and resilience...

The policy takes the form of a `Predicate` of some type. A predicate is useful since we can just filter the list of courses in the course plan. That being said, you will still need to think about an appropriate *form* of the predicate.

Write each of the policies above as separate classes `SUstart` and `SUend`. Here are some examples:

```
jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113)).
...> trimSU(new SUstart())
$.. ==> InfList$.

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113)).
...> trimSU(new SUstart()). // cs1010 at start of the chain is SU'ed
...> forEach(x -> System.out.println(x))
[cs2030]
[cs2040]
[cs2113]

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040, cs2113)).
...> trimSU(new SUend()). // cs2113 at end of the chain is SU'ed
...> forEach(x -> System.out.println(x))
[cs1010]
[cs2030]
[cs2040]

jshell> new CoursePlan(InfList.of(cs1010, cs2030, cs2040)).
...> trimSU(new SUend()). // cs2030 and cs2040 at the end of both chains are SU'ed
...> forEach(x -> System.out.println(x))
[cs1010]
```

In the last case, the student who does not do well in both CS2030 and CS2040, will S/U the courses and

most likely change to another degree programme without incurring too much harm to his/her GPA.

Your Files:

[SUBMIT](#) (only .java, .c, .cpp, .h, .jsh, and .py extensions allowed)

To submit multiple files, click on the Browse button, then select one or more files. The selected file(s) will be added to the upload queue. You can repeat this step to add more files. Check that you have all the files needed for your submission. Then click on the Submit button to upload your submission.